Politechnika Warszawska

WYDZIAŁ MECHATRONIKI



Praca dyplomowa magisterska

na kierunku Automatyka, Robotyka i Informatyka Przemysłowa

Projekt i implementacja schedulera systemów operacyjnych o złożoności O(1)

Kajetan Kontek

numer albumu 327749

promotor

dr inż. Bartłomiej Fajdek

WARSZAWA 2027

Streszczenie

Modułowy szablon pracy dyplomowej w systemie LaTeX

Wytyczne Rektora Politechniki Warszawskiej precyzują kilka rygorystycznych wymogów odnośnie streszczenia pracy dyplomowej. Streszczenie stanowi odrębny punkt w układzie pracy i musi znajdować się na **osobnej stronie**. Obowiązuje tu również **ścisła objętość i formatowanie**, ponieważ każde streszczenie, niezależnie od języka, musi zajmować dokładnie jedną stronę, przy czym należy w nim zastosować pojedynczy odstęp (interlinię) oraz czcionkę o rozmiarze 12 pkt.

Jeśli chodzi o **wymagane elementy**, to poza samym tekstem podsumowującym pracę, w każdym streszczeniu obowiązkowo musi pojawić się tytuł pracy oraz zestaw słów kluczowych. W praktyce akademickiej jako zestaw słów kluczowych przyjmuje się zazwyczaj od 3 do 5 pojęć lub krótkich fraz, które najdokładniej oddają tematykę i zakres przeprowadzonych w pracy badań. Umieszcza się je pod tekstem streszczenia.

Same **wersje językowe** zależą bezpośrednio od tego, w jakim języku napisana jest cała praca. Dla pracy w języku polskim wymagane jest osobne streszczenie po polsku oraz drugie streszczenie po angielsku. Dla pracy w języku angielskim wymagane jest streszczenie po angielsku oraz streszczenie po polsku (w tej kolejności). Z kolei dla pracy w innym języku obcym wymagane jest streszczenie w języku, w którym napisano pracę, a ponadto streszczenie po polsku oraz streszczenie po angielsku. **Opcjonalnie**, jeśli zamierzasz wystąpić o wydanie odpisu dyplomu w tłumaczeniu na jeszcze inny język obcy, powinieneś dołączyć dodatkowe streszczenie napisane w tym właśnie języku.

Warto zadbać o to, aby styl pisania był zwięzły i odzwierciedlał strukturę pracy: cel, metody, wyniki i wnioski.

Słowa kluczowe: praca dyplomowa, układ modułowy, LaTeX

Abstract

Modular diploma thesis template in LaTeX system

Here goes the English version of your abstract. It must follow the exact same formatting rules: 12pt font size, single line spacing, and it must fit entirely on this single page.

Keywords: diploma thesis, modular layout, LaTeX

Spis treści

1 Istniejące algorytmy planowania czasu procesora w nowoczesnych systemach 9
1.1 System Linux i architektura EEVDF 9
1.1.1 Wybór procesu i struktury danych 10
1.1.2 Mechanizm Run to Parity 10
1.2 Systemy z grupy Windows 10
1.2.1 Wybór procesu i struktury danych 11
1.3 Systemy z rodziny BSD na przykładzie FreeBSD 12
1.3.1 Wybór procesu i struktury danych 13

2 Projekt i realizacja systemu 15

3 Wyniki badań 16

4 Podsumowanie 17

Literatura 18

Wykaz symboli i skrótów 21

Spis załączników 27

1. Istniejące algorytmy planowania czasu procesora w nowoczesnych systemach

1.1 System Linux i architektura EEVDF

W nowoczesnych wersjach jądra systemu Linux, domyślnym algorytmem planowania (ang. *scheduler*) dla procesów standardowych jest algorytm Earliest Eligible Virtual Deadline First (EEVDF). Zastąpił on wieloletni standard znany jako Completely Fair Scheduler (CFS), rozwiązując fundamentalny problem balansu między sprawiedliwością przydziału czasu a opóźnieniami w obsłudze zadań interaktywnych [1, 2].

Działanie algorytmu EEVDF opiera się na ścisłych podstawach matematycznych [3] i operuje na kilku kluczowych wartościach przypisywanych każdemu zadaniu:

- **Żądany kwant czasu** (*requested time slice / slice*)
- **Wirtualny czas wykonania** (*virtual runtime / vruntime*)
- **Opóźnienie** (*lag*)
- **Priorytet** (*priority / weight*)

Kiedy nowe lub powracające ze stanu uśpienia zadanie trafia do kolejki procesów gotowych, otrzymuje bazowy kwant czasu (*slice*). Wartość ta oparta jest na zmiennych konfiguracyjnych systemu (np. `sysctl_sched_base_slice`) i jest dynamicznie skalowana na podstawie wagi procesu (wynikającej z jego priorytetu *nice*) [4, 5]. Następnie, gdy zadanie zaczyna być wykonywane przez procesor, jego wirtualny czas wykonania (`vruntime`) stale rośnie. Wartość ta odzwierciedla rzeczywisty czas spędzony na procesorze, znormalizowany przez wagę zadania. Zliczanie tego czasu opiera się na precyzyjnych licznikach sprzętowych procesora (np. w architekturze amd64), jednak sama aktualizacja wirtualnego czasu i struktur danych jest w całości realizowana programowo przez jądro systemu operacyjnego, zazwyczaj podczas obsługi przerwań zegarowych [6, 4].

Na bazie początkowo przypisanego kwantu czasu oraz `vruntime`, system wylicza wirtualny termin zakończenia (*virtual deadline*). Reprezentuje on moment, do którego proces powinien w idealnych warunkach skonsumować swój przydzielony czas. Algorytm na bieżąco oblicza również opóźnienie (*lag*), będące różnicą pomiędzy czasem, jaki zadanie powinno było otrzymać w idealnym, proporcjonalnym modelu podziału czasu (tzw. *fluid-flow model*), a czasem, który faktycznie wykorzystało. Jeżeli wartość *lag* jest mniejsza od zera, oznacza to, że proces otrzymał więcej czasu procesora niż pozostałe procesy rywalizujące o czas procesora. W takim scenariuszu proces traci kwalifikowalność (*ineligible*) i jest kategorycznie wykluczony z możliwości przydziału sprzętowego do momentu, aż ogólny upływ czasu w systemie sprawi, że jego *lag* ponownie wzrośnie do wartości nieujemnej [1, 3].

1.1.1 Wybór procesu i struktury danych

Działanie każdego planisty w nowoczesnych systemach operacyjnych opiera się na sprzętowym mechanizmie przerwań czasowych (ang. *timer interrupts*). Ponieważ procesor nie może jednocześnie wykonywać własnego kodu schedulera oraz kodu aplikacji użytkownika, wbudowany kontroler przerwań z dużą częstotliwością wstrzymuje wykonywane zadanie, przekazując sterowanie do jądra systemu [7]. Daje to algorytmowi możliwość przeliczenia wskaźników i, w razie potrzeby, wywłaszczenia procesu.

Scheduler EEVDF przechowuje informacje o aktywnych procesach w zrównoważonym drzewie czerwono-czarnym (ang. *Augmented Red-Black Tree*). W przeciwieństwie do naiwnego podejścia, drzewo to nadal sortuje węzły na podstawie wirtualnego czasu wykonania (`vruntime`), co układa zadania od lewej do prawej. Kluczową innowacją jest jednak rozszerzenie (*augmentation*) każdego węzła o dodatkowe pole `min_deadline`, które przechowuje najkrótszy termin zakończenia z danego węzła i wszystkich jego potomków. Dzięki temu struktura działa jednocześnie jako drzewo wyszukiwania binarnego oraz jako kopiec (min-heap) dla wartości *deadline*. Architektura ta pozwala systemowi w optymalnym czasie $O(\log n)$ odrzucić całe poddrzewa i natychmiast odnaleźć zadanie o najwcześniejszym *virtual deadline*, które jednocześnie spełnia warunek kwalifikowalności ($lag \geq 0$) i to ono zostaje wybrane do działania na procesorze [8, 4].

1.1.2 Mechanizm Run to Parity

Algorytm EEVDF w swojej czystej, teoretycznej postaci zakłada wyidealizowany model maszyny obliczeniowej. W rzeczywistości fizyczne procesory opierają swoją wydajność na hierarchii pamięci podręcznej (Cache L1/L2/L3), która przechowuje dane powiązane z aktualnie wykonywanym procesem. Niezmodyfikowany mechanizm doboru zadań oparty wyłącznie na *deadline'ach* prowadziłby do zjawiska tzw. *over-scheduling* – zbyt częstych zmian aktualnie działającego zadania. Skutkowałoby to znacznymi stratami cykli procesora na ciągłą wymianę linii pamięci podręcznej (*cache miss*) oraz narzutem związanym z przełączaniem kontekstu.

Aby rozwiązać ten problem, w systemie Linux wprowadzono mechanizm *Run to Parity* [5]. Zgodnie z tą regułą, proces, który już przejął kontrolę nad jednostką obliczeniową, z reguły nie zostaje wywłaszczony przez inne, standardowe zadania posiadające wcześniejszy *virtual deadline*, dopóki jego własne opóźnienie (*lag*) nie spadnie do zera. Dopiero gdy proces "wyrówna" swój dług wobec systemu (osiągając parytet), zdejmuje się z niego ochronę, pozwalając na przejęcie procesora przez kolejne kwalifikujące się zadanie. Mechanizm ten gwarantuje dłuższe, nieprzerwane odcinki pracy pojedynczego wątku, co maksymalizuje wykorzystanie pamięci podręcznej i zwiększa ogólną przepustowość sprzętu (throughput), nie naruszając jednocześnie długoterminowych matematycznych gwarancji sprawiedliwości algorytmu [2].

1.2 Systemy z grupy Windows

W nowoczesnych systemach z rodziny Windows za przydział czasu procesora odpowiada planista (ang. *scheduler*) zaprojektowany pierwotnie na potrzeby architektury Microsoft Windows NT. W przeciwieństwie do algorytmów opartych na sprawiedliwym podziale czasu, rozwiązanie firmy Microsoft bazuje na wywłaszczeniowym planowaniu wielozadaniowym, w którym absolut-

nym fundamentem jest rygorystyczny priorytet [9]. Co istotne, jednostką planowania w systemie Windows nie jest proces, lecz pojedynczy wątek. Proces pełni jedynie rolę kontenera na zasoby (takie jak pamięć czy kontekst zabezpieczeń), podczas gdy o dostęp do jednostki obliczeniowej rywalizują bezpośrednio wątki [10].

Architektura planisty definiuje 32 odrębne poziomy priorytetów (od 0 do 31), które podzielone są na trzy główne kategorie [11]:

- **Poziom 0 (Zero-Page Thread):** Zarezerwowany wyłącznie dla systemowego wątku bezczynności (*idle thread*), który uruchamia się tylko przy braku innych zadań. Zajmuje się on m.in. asynchronicznym zerowaniem wolnych stron pamięci fizycznej.
- **Poziomy 1–15 (Priorytety dynamiczne):** Przydzielane standardowym wątkom aplikacji użytkownika oraz niekrytycznym wątkom jądra. Podlegają one ciągłym, dynamicznym algorytmom modyfikacji (wzmacnianiu i degradacji).
- **Poziomy 16–31 (Priorytety czasu rzeczywistego):** Zarezerwowane dla absolutnie krytycznych operacji (np. przetwarzanie multimediów przy minimalnych opóźnieniach, sterowniki sprzętowe). Wątki na tych poziomach mają sztywny priorytet i nie podlegają dynamicznym zmianom [12].

Przy tworzeniu wątku w paśmie dynamicznym przydzielana jest mu tzw. wartość bazowa priorytetu (*base priority*), obliczana na podstawie klasy priorytetu procesu oraz priorytetu wątku wewnątrz tej klasy. Aby zapewnić wysoką responsywność interaktywną i zapobiegać zbyt długiemu oczekiwaniu zadań (*starvation*), planista stosuje mechanizm dynamicznego wzmacniania priorytetu (*priority boosting*). W systemie istnieje sześć głównych rodzajów zdarzeń, które wyzwalają tymczasowe podbicie priorytetu wątku [13]:

1. Zakończenie asynchronicznej operacji wejścia/wyjścia (I/O completion).
2. Wybudzenie wątku przez obiekty synchronizacji (np. semafory, zdarzenia).
3. Zakończenie oczekiwania na zasoby warstwy wykonawczej (Executive resources).
4. Działanie w ramach procesu pierwszoplanowego (Foreground process).
5. Wybudzenie w celu obsługi zdarzeń interfejsu graficznego (GUI windowing activity).
6. Interwencja usługi *Balance Set Manager*, chroniąca przed głodzeniem poprzez wywindowanie priorytetu zablokowanego wątku do wartości 15 [14].

Po otrzymaniu wzmocnienia, kwant czasu (ang. *quantum*) wątku ulega wyzerowaniu. Kiedy wątek w pełni wykorzysta nowo przydzielony kwant, system stosuje mechanizm degradacji (*priority decay*) – jego priorytet jest zmniejszany o 1. Proces ten powtarza się przy każdym kolejnym wyczerpaniu kwantu, aż wartość spadnie z powrotem do nominalnego priorytetu bazowego [13].

1.2.1 Wybór procesu i struktury danych

Kernel utrzymuje niezależne struktury danych dla każdego procesora logicznego, zgrupowane wewnątrz bloku *Kernel Processor Control Block* (KPRCB) [15]. Serce tej struktury stanowią

kolejki gotowości (*Dispatcher Ready Queues*), zaimplementowane jako tablica 32 dwukierunkowych list powiązanych – po jednej dla każdego poziomu priorytetu. Nowe lub wybudzone wątki dodawane są na koniec listy odpowiadającej ich priorytetowi.

W celu optymalizacji wyboru następnego zadania, dyspozytor unika czasochłonnego iterowania po listach. Utrzymuje on pojedynczą, 32-bitową liczbę, zwaną *Ready Summary*. Każdy bit tej zmiennej powiązany jest z jedną kolejką. Jeśli dana kolejka posiada przynajmniej jeden wątek gotowy do natychmiastowego uruchomienia, odpowiadający jej bit ustawiany jest na wartość 1. Jeżeli kolejka jest pusta, bit przyjmuje wartość 0 [11].

Dzięki takiej architekturze, problem wyboru następnego wątku sprowadza się do odnalezienia najstarszego ustawionego bitu (ang. *most significant set bit*) w zmiennej *Ready Summary*. Jest to operacja matematyczna, której złożoność czasowa wynosi $O(1)$. W architekturach sprzętowych z rodziny x86/amd64 poszukiwanie to realizowane jest za pomocą pojedynczej instrukcji procesora – BSR (*Bit Scan Reverse*). Pozwala to w bardzo krótkim czasie zidentyfikować najwyższą niepustą kolejkę, pobrać z niej pierwszy oczekujący wątek i zainicjować przełączenie kontekstu [16].

1.3 Systemy z rodziny BSD na przykładzie FreeBSD

W nowoczesnych wersjach systemu FreeBSD (począwszy od wydania 5.2) domyślnym podsystemem szeregowania zadań jest planista ULE [17]. Zastąpił on historyczny algorytm wywodzący się z systemu 4.4BSD, który ze względu na globalną kolejkę uruchomieniową oraz konieczność okresowego przeliczania priorytetów wszystkich zadań w systemie (złożoność $O(n)$) nie skalował się efektywnie w środowiskach wieloprocesorowych (SMP) [17, 18]. Nazwa ULE nie jest akronimem – pochodzi od trzech ostatnich liter angielskiego słowa *schedule* (powstałego po usunięciu znaku podkreślenia z nazwy pliku źródłowego `sched_ule.c`) [17, 19]. Analogicznie jak w systemie Windows, podstawową jednostką podlegającą szeregowaniu we FreeBSD nie jest cały proces (`struct proc`), lecz pojedynczy wątek jądra (`struct thread`), posiadający własną strukturę metadanych planisty `struct td_sched` [19].

Architektura FreeBSD definiuje 256 dyskretnych poziomów priorytetów (od 0 do 255). Zgodnie z klasyczną konwencją systemów UNIX, niższa wartość liczbowa oznacza wyższy priorytet wykonywania [18]. Przestrzeń ta podzielona jest na cztery główne klasy [18]:

- **Poziomy 0–47 (Wątki przerwań i jądra):** Najwyższe priorytety nadawane podsystemom jądra, gwarantujące bezwzględne prawo wywłaszczania procesów przestrzeni użytkownika.
- **Poziomy 48–79 (Wątki czasu rzeczywistego):** Zadania o krytycznym znaczeniu czasowym (*real-time*), których priorytety są nadawane bezpośrednio przez aplikacje i nie podlegają kwantowaniu czasu.
- **Poziomy 120–223 (Wątki podziału czasu):** Pasmo przeznaczone dla standardowych procesów użytkownika (*timesharing*), w którym priorytet wyliczany jest dynamicznie przez planistę.
- **Poziomy 224–255 (Wątki bezczynności):** Zarezerwowane dla zadań wykonywanych wyłącznie przy braku jakichkolwiek innych gotowych wątków w systemie (*idle threads*).

W momencie utworzenia nowego procesu za pomocą wywołania `fork()`, wątek potomny dziedziczy początkowy poziom priorytetu oraz skumulowaną historię interaktywności bezpośrednio po swoim procesie macierzystym (np. powłoce systemowej) [17]. Sprawia to, że nowo powołane zadania zazwyczaj rozpoczynają pracę z wysokim priorytetem charakterystycznym dla aplikacji interaktywnych. W trakcie dalszego działania wątku w paśmie podziału czasu (120–223), jego priorytet ulega ciągłym zmianom w zależności od zachowania zadania oraz przypisanego mu przez użytkownika parametru *nice* (z przedziału od -20 do +20) [17, 18].

Kluczowym mechanizmem modyfikującym priorytet w czasie pracy jest heurystyka oceny interaktywności, realizowana przez funkcję `sched_interact_score()` [19]. Planista monitoruje w strukturze `td_sched` skumulowany czas dobrowolnego uśpienia wątku (T_{sleep}) oraz czas jego faktycznej pracy na procesorze (T_{run}) w obrębie ruchomego okna historii (ograniczonego stałą `SCHED_SLP_RUN_MAX`) [17, 19]. Oznaczając przez M połowę maksymalnej skali interaktywności, wskaźnik S wyznaczany jest zależnie od relacji obu czasów. Gdy czas snu przewyższa czas pracy ($T_{\text{sleep}} > T_{\text{run}}$), wskaźnik przyjmuje postać [17]:

$$S = \left\lfloor \frac{M \cdot T_{\text{run}}}{T_{\text{sleep}}} \right\rfloor$$

Natomiast gdy dominuje czas obliczeń ($T_{\text{run}} \geq T_{\text{sleep}}$), wartość obliczana jest ze wzoru [17]:

$$S = M + \left\lfloor \frac{M \cdot (T_{\text{run}} - T_{\text{sleep}})}{T_{\text{run}}} \right\rfloor$$

Im niższa jest wyliczona wartość S, tym wyższą interaktywnością cechuje się zadanie. Jeżeli wskaźnik S spadnie poniżej zdefiniowanego progu, wątek zostaje uznany za interaktywny, a jego priorytet jest wyznaczany proporcjonalnie do wyniku interaktywności (z uwzględnieniem modyfikatora *nice*) [17]. Jeśli jednak proces (np. kompilator uruchomiony z interaktywnej powłoki) zacznie intensywnie obciążać jednostkę obliczeniową bez dobrowolnego zasypiania, wartość T_{run} szybko zdominuje T_{sleep}. Przekroczenie progu interaktywności spowoduje degradację zadania do kategorii wsadowej (*batch*). Dla takich wątków priorytet wewnątrz pasma podziału czasu rośnie (czyli staje się mniej korzystny) wraz z liczbą wyczerpywanych taktów zegara statystycznego (`statclock`) oraz dodatnią wartością *nice* [17, 18].

1.3.1 Wybór procesu i struktury danych

Aby wyeliminować rywalizację o blokady między rdzeniami i osiągnąć złożoność czasową $O(1)$, scheduler ULE utrzymuje osobny zestaw kolejek uruchomieniowych `struct tdq` dla każdego procesora logicznego (w tablicy `tdq_cpu`) [19]. Wewnątrz każdego kontenera (zestawu) `tdq` znajdują się trzy wyspecjalizowane struktury `struct runq`:

1. `tdq_realtime` – przechowuje wątki przerwań, zadania czasu rzeczywistego oraz wątki podziału czasu zaklasyfikowane jako interaktywne [18, 19].

2. `tdq_timeshare` – gromadzi nieinteraktywne (wsadowe) wątki podziału czasu. Działa ona jako kolejka kalendarzowa (*calendar queue*), wykorzystująca dwa rotujące indeksy: zapisu (`tdq_idx`) oraz odczytu (`tdq_ridx`). Wstawiane wątki umieszczane są w odległości od `tdq_idx` zależnej od ich priorytetu, natomiast pobieranie odbywa się od pozycji `tdq_ridx`, która przesuwa się z upływem czasu. Gwarantuje to, że każde zadanie wsadowe zosta-

nie uruchomione podczas pełnego obrotu wskaźnika, co całkowicie zapobiega zjawisku głodzenia (*starvation*) bez potrzeby globalnego przeliczania priorytetów [18, 19].

3. `tdq_idle` – gromadzi wątki bezczynności, uruchamiane wyłącznie wtedy, gdy dwie powyższe struktury są całkowicie puste [19].

Na najniższym poziomie każda z trzech instancji `struct runq` agreguje 256 poziomów priorytetów w 64 dwukierunkowe listy ogonowe (`rq_queues[64]`). Samo odnalezienie niepustej kolejki wewnątrz wybranej struktury realizowane jest w czasie stałym $O(1)$ przez funkcję `runq_choose()`, która odczytuje 64-bitowy wektor stanu `rq_status`. Pozycja pierwszego ustawionego bitu bezpośrednio wskazuje indeks najważniejszej niepustej listy, z której pobierany jest pierwszy oczekujący wątek [18, 20]. Jest to bardzo podobne rozwiązanie optymalizacyjne jak w systemach z rodziny Windows.

2. Projekt i realizacja systemu

Niniejszy rozdział stanowi zasadniczą część merytoryczną pracy dyplomowej. Autor szczegółowo opisuje w nim strukturę projektową urządzeń, układów lub algorytmów oraz ich praktyczną realizację.

Treść powinna w sposób ustrukturyzowany dokumentować proces realizacji projektu od założeń wstępnych do efektu końcowego, uwzględniając:

- opracowanie koncepcji oraz uzasadnienie wyboru stosowanych metod, narzędzi, technologii lub elementów składowych,
- dokumentację projektową i inżynierską (np. schematy, modele matematyczne, diagramy blokowe, algorytmy),
- przebieg prac implementacyjnych, realizacyjnych lub symulacyjnych oraz integrację poszczególnych modułów i podsystemów.

Stylistyka: tekst należy redagować w bezosobowej formie czasu przeszłego (np. *„zaprojektowano”, „zaimplementowano”, „przeprowadzono dobór”*), zachowując charakter sprawozdania technicznego.

3. Wyniki badań

Niniejszy rozdział służy prezentacji, eksperymentalnej walidacji oraz krytycznej analizie uzyskanych wyników badawczych, pomiarowych lub symulacyjnych.

Treść powinna w sposób obiektywny dokumentować działanie i charakterystykę opracowanego rozwiązania, uwzględniając:

- opis metodyki badawczej, warunków przeprowadzenia eksperymentu lub scenariuszy testowych,
- opracowane i czytelnie sformatowane dane wynikowe zaprezentowane w formie tabel zbiorczych oraz wykresów,
- interpretację uzyskanych zależności, analizę błędów pomiarowych oraz odniesienie wyników do założeń początkowych i stanu wiedzy.

Stylistyka: w tym rozdziale nie należy używać określeń subiektywnych (np. *„wyniki są bardzo dobre”, „system działa świetnie”*). Należy posługiwać się wyłącznie językiem faktów, liczb i konkretnych wskaźników inżynierskich, stosując bezosobową formę czasu przeszłego.

4. Podsumowanie

Niniejszy rozdział stanowi formalne zamknięcie merytorycznej części pracy dyplomowej. Jego celem jest zwięzła synteza uzyskanych rezultatów oraz globalna ocena stopnia realizacji zadania inżynierskiego.

W rozdziale tym należy dokonać całościowej oceny zrealizowanych prac inżynierskich, opierając strukturę wniosków na następujących kluczowych aspektach:

- jednoznacznej weryfikacji stopnia realizacji celów oraz tez sformułowanych we wstępie pracy,
- zwięzłym omówieniu najważniejszych osiągnięć o charakterze konstrukcyjnym, programistycznym bądź eksperymentalnym,
- obiektywnej ocenie ograniczeń wypracowanego rozwiązania oraz wskazaniu potencjalnych kierunków jego dalszej optymalizacji lub rozbudowy.

Stylistyka: wnioski powinny być sformułowane w sposób konkretny i zwięzły. Należy unikać dosłownego powtarzania zdań z poprzednich rozdziałów, skupiając się na ogólnej konkluzji z całej pracy.

Więcej informacji na temat systemu LaTeX:

- https://www.overleaf.com/learn – bardzo przystępny, oficjalny tutorial i baza wiedzy na stronie platformy Overleaf,
- https://www.latex-project.org/ – oficjalna strona domowa projektu LaTeX, zawierająca aktualności i dokumentację standardów,
- https://www.tug.org/begin.html – prowadzony przez *TeX Users Group* obszerny zbiór odnośników do materiałów dla osób początkujących,
- https://www.texstudio.org/ – oficjalna strona domowa edytora TeXstudio, z której należy pobierać najnowsze, stabilne wersje programu,
- https://www.xm1math.net/texmaker/ – oficjalna strona domowa edytora Texmaker, z której można pobrać program oraz uzyskać dostęp do jego podręcznika użytkownika,
- https://texstudio-org.github.io/ – oficjalny, szczegółowy podręcznik użytkownika (*User Manual*) wyjaśniający zaawansowane funkcje edytora TeXstudio,
- https://tex.stackexchange.com/ – największe na świecie forum społecznościowe typu Q&A (pytania i odpowiedzi), na którym można znaleźć rozwiązanie niemal każdego problemu technicznego i błędu kompilacji.

Literatura

[1] The Linux Kernel Developers. *EEVDF Scheduler – The Linux Kernel documentation*. 2024. URL: https://docs.kernel.org/scheduler/sched-eevdf.html (term. wiz. 10. 10. 2026).

[2] Ingo Molnar. *[GIT PULL] Scheduler changes for v6.12*. 2024. URL: https://lkml.org/lkml/2024/9/19/397 (term. wiz. 10. 10. 2026).

[3] Ion Stoica i Hussein Abdel-Wahab. "Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation". W: *Technical Report, Old Dominion University* (1995). URL: https://www.semanticscholar.org/paper/Earliest-Eligible-Virtual-Deadline-First-%3A-A-and-Stoica-Abdel-Wahab/9d07417330e3d37a5767359531b4adc4c27a5349.

[4] Linus Torvalds i contributors. *linux/kernel/sched/fair.c at master*. 2024. URL: https://github.com/torvalds/linux/blob/master/kernel/sched/fair.c (term. wiz. 10. 10. 2026).

[5] Peter Zijlstra. *[sched/core] sched: Move PLACE_LAG and RUN_TO_PARITY to sysctl*. 2025. URL: https://lkml.org/lkml/2025/2/11/1991 (term. wiz. 10. 10. 2026).

[6] Linus Torvalds i contributors. *linux/include/linux/sched.h at master*. 2024. URL: https://github.com/torvalds/linux/blob/master/include/linux/sched.h (term. wiz. 10. 10. 2026).

[7] Wikipedia contributors. *Scheduling (computing) – Wikipedia, The Free Encyclopedia*. 2026. URL: https://en.wikipedia.org/wiki/Scheduling_(computing) (term. wiz. 10. 10. 2026).

[8] Linux Kernel Internals. *EEVDF – Linux Kernel Internals*. 2024. URL: https://kernel-internals.org/sched/eevdf/ (term. wiz. 10. 10. 2026).

[9] Loyola University Chicago, Department of Computer Science. *Process/Thread Scheduling – Introduction to Operating Systems: Principles and Practice*. 2026. URL: https://os.cs.luc.edu/scheduling.html (term. wiz. 11. 10. 2026).

[10] Microsoft. *About Processes and Threads*. 2025. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/about-processes-and-threads (term. wiz. 11. 10. 2026).

[11] David A. Solomon i Mark E. Russinovich. "Thread Scheduling". W: *Inside Microsoft Windows 2000*. 3 wyd. Microsoft Press, 2000. ISBN: 0735610215. URL: https://flylib.com/books/en/3.169.1.54/1/ (term. wiz. 11. 10. 2026).

[12] Microsoft. *Priority Boosts*. 2025. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/priority-boosts (term. wiz. 11. 10. 2026).

[13] BSODTutorials. *Thread Quantum, Thread Priority Boosting and Processor Affinity*. 2013. URL: http://bsodtutorials.blogspot.com/2013/11/thread-quantum-thread-priority-boosting.html (term. wiz. 11. 10. 2026).

[14] Joe Duffy. *Priority-induced starvation: Why Sleep(1) is better than Sleep(0) and the Windows balance set manager*. 2006. URL: https://joeduffyblog.com/2006/08/22/priorityinduced-starvation-why-sleep1-is-better-than-sleep0-and-the-windows-balance-set-manager/ (term. wiz. 11. 10. 2026).

[15] ReactOS Project. *ntoskrnl/ke/thrdschd.c Source File*. ReactOS 0.4.17-dev. 2026. URL: https://doxygen.reactos.org/db/d73/thrdschd_8c_source.html (term. wiz. 11. 10. 2026).

[16] ReactOS Project. *ntoskrnl/include/internal/ke_x.h File Reference*. ReactOS 0.4.17-dev. 2026. URL: https://doxygen.reactos.org/de/d90/ke__x_8h.html (term. wiz. 11. 10. 2026).

[17] Jeff Roberson. "ULE: A Modern Scheduler for FreeBSD". W: *Proceedings of the BSD Conference 2003 (BSDCon '03)*. USENIX Association, 2003, s. 17–28. URL: https://www.usenix.org/legacy/event/bsdcon03/tech/full_papers/roberson/roberson.pdf (term. wiz. 15. 10. 2026).

[18] Marshall Kirk McKusick, George V. Neville-Neil i Robert N. M. Watson. "Thread Scheduling". W: *The Design and Implementation of the FreeBSD Operating System*. 2 wyd. Addison-Wesley Professional, 2014. Rozd. 4.4. ISBN: 978-0321968975. URL: https://www.informit.com/articles/article.aspx?p=2249436&seqNum=4 (term. wiz. 15. 10. 2026).

[19] Jeff Roberson i The FreeBSD Project. *sys/kern/sched_ule.c – FreeBSD Kernel Source Tree*. 2026. URL: https://cgit.freebsd.org/src/tree/sys/kern/sched_ule.c (term. wiz. 15. 10. 2026).

[20] The FreeBSD Project. *sys/sys/runq.h – FreeBSD Kernel Source Tree*. 2026. URL: https://cgit.freebsd.org/src/tree/sys/sys/runq.h (term. wiz. 15. 10. 2026).

Wykaz symboli i skrótów

ω	prędkość kątowa [rad/s]
M	moment obrotowy [Nm]
U	napięcie zasilania [V]
I	natężenie prądu [A]
θ	kąt obrotu wału silnika [$^\circ$ lub rad]
F	siła ciągu/nacisku [N]
k_p, k_i, k_d	nastawy (wzmocnienia) regulatora PID

ADC	(ang. *Analog-to-Digital Converter*) przetwornik analogowo-cyfrowy
DOF	(ang. *Degrees of Freedom*) stopnie swobody
IoT	(ang. *Internet of Things*) internet rzeczy
PLC	(ang. *Programmable Logic Controller*) programowalny sterownik logiczny

Spis rysunków

Spis tabel

Spis załączników

1. Analiza błędów – pełny skrypt źródłowy zawierający zaimplementowane funkcje testowe oraz procedurę numerycznej analizy błędów metody trapezów.
2. Specyfikacja techniczna stacji roboczej wykorzystanej w pracy.
3. Dodatkowe zdjęcia stanowiska badawczego.

Załącznik 1
Analiza błędów

Zarządzenie Rektora PW ściśle definiuje układ dokumentu, wskazując, że przedostatnim jego elementem jest „Spis załączników”, w którym wymienia się te materiały, które są fizycznie dołączone na samym końcu dokumentu pracy dyplomowej.

Poniżej znajduje się pełny skrypt zawierający funkcje testowe oraz procedurę analizy błędów.

Załącznik 2
Specyfikacja techniczna stacji roboczej wykorzystanej w pracy

Komponent	**Parametry techniczne / Model**
Procesor (CPU)	Intel Core i7-14700K (20 rdzeni, 28 wątków, taktowanie do 5.6 GHz)
Pamięć RAM	64 GB DDR5 Kingston Fury (2 $\times$ 32 GB, 6000 MHz, CL30)
Karta graficzna (GPU)	NVIDIA GeForce RTX 4080 Super (16 GB GDDR6X, 10240 rdzeni CUDA)
Dysk twardy	SSD Samsung 990 Pro 2 TB NVMe M.2 PCIe Gen4 (odczyt do 7450 MB/s)
Płyta główna	MSI MAG Z790 TOMAHAWK WIFI
Zasilacz	Corsair RM850x 850W (Certyfikat 80 Plus Gold)
System operacyjny	Linux Ubuntu 24.04 LTS / Windows 11 Pro (64-bit)
Środowisko i biblioteki	Python 3.12, CUDA Toolkit 12.4, PyTorch 2.3, MATLAB R2024a

Załącznik 3
Dodatkowe zdjęcia stanowiska badawczego

W niniejszym załączniku zgromadzono dodatkowe materiały graficzne, dokumentację fotograficzną stanowiska badawczego oraz szczegółowe wykresy przebiegów, które ze względu na swoje znaczne rozmiary i pomocniczy charakter nie zostały włączone bezpośrednio do zasadniczej części pracy. Zamieszczone poniżej materiały stanowią bezpośrednie uzupełnienie opisów eksperymentów zawartych w rozdziale 3 i umożliwiają pełniejszą, szczegółową weryfikację poprawności działania zaimplementowanego systemu.